

NEXUS AI

Agent & Tool System Document

N37–N45 • Agents • Planning • Tools • Filesystem • Terminal • Coding • Sandbox • State • Multi-Agent

Version: 0.1 **Status:** Agent Engineering Foundation **Security Model:** Least Privilege

Purpose: Define how NEXUS converts user goals into bounded agent tasks and controlled tool operations. The model proposes reasoning and actions; NEXUS owns orchestration, permissions, execution and verification.

USER GOAL → PLAN → CONTEXT → AGENT STEP → TOOL REQUEST → POLICY → EXECUTION → OBSERVATION →
VERIFY → COMPLETE

1. Agent System Vision

NEXUS agents are bounded task workers, not unrestricted autonomous processes. An agent has a role, goal, context, tools, permissions, budget, state and completion criteria. The Agent Runtime coordinates these elements and prevents a language model from directly controlling the operating system.

Core Principles

- Agent plans are bounded and observable.
- Models request tool operations; the Tool Gateway decides whether they execute.
- Every task has a maximum step/retry/tool budget.
- High-impact actions require confirmation or explicit policy approval.
- Agent state is separate from raw conversation history.
- Tool results are treated as untrusted observations.
- Verification is part of completion for important tasks.
- Agents must stop when blocked, unsafe or outside scope.
- Multi-agent delegation is explicit and bounded.

2. N37 — Agent System Specification

An Agent Definition describes what an agent is allowed to do and how it completes work.

Agent Property	Purpose
Agent ID	Stable identifier.
Role	Planner, coder, researcher, tester, reviewer, verifier, etc.
Goal	Task objective supplied by orchestrator.
System Instructions	Role-specific behavior and constraints.
Model Policy	Required capabilities and eligible model class.
Tool Set	Allowlisted tools.
Permission Set	Filesystem, command and data access.
Context Policy	Conversation, project, memory and RAG sources allowed.
Budget	Maximum tokens, time, steps and tool calls.
Stop Conditions	Success, failure, timeout, approval or budget exhaustion.
Verification Policy	Required checks before completion.
State Schema	Structured progress and observations.

Agent Lifecycle

CREATED → PLANNED → READY → RUNNING → WAITING_FOR_TOOL/USER → VERIFYING → COMPLETED / FAILED / CANCELLED.

Agent Status Rules

Only the orchestrator may transition task state. Tools return observations; they do not directly mark an agent complete. Completion requires the configured success criteria and verification policy.

3. N38 — Agent Planning & Execution Architecture

Planning is separated from execution. A simple request may skip multi-step planning, while complex tasks use a planner to produce a bounded plan.

Goal → Intent Analysis → Plan → Validate Plan → Execute Step → Observe → Update State → Next Step → Verify → Finalize.

Stage	Responsibility
Intent	Identify objective, constraints and expected output.
Planning	Break objective into minimal useful steps.
Plan validation	Check tool availability, permissions and budget.
Execution	Invoke model and approved tools.
Observation	Record tool/model results.
Replanning	Change remaining steps if observations invalidate the plan.
Verification	Check completion against acceptance criteria.
Finalization	Return result, changes, evidence and unresolved issues.

Planning Rules

Plans must avoid unnecessary steps. The planner should prefer inspection before modification, tests before claiming success, and read-only actions before write actions. Plans must contain explicit completion criteria.

Execution Budget

Every task has configurable limits: maximum wall-clock time, agent steps, model calls, tool calls, retries, output tokens and filesystem modifications.

4. N39 — Tool Registry Specification

The Tool Registry is the authoritative list of capabilities that agents can request.

Tool Category	Initial Tools	Risk
Filesystem	list, search, read, write/edit	Low–Medium
Terminal	run approved command	High
Testing	run test/lint/build command	Medium
Git	status, diff, log; controlled writes later	Medium
Knowledge	search RAG, retrieve source	Low
System	system/resource information	Low
Future	browser, GitHub, package manager, integrations	Variable

Tool Definition Contract

Each tool defines: name, version, description, input schema, output schema, permission scope, risk level, timeout, resource limits, approval requirement, audit fields and failure codes.

Tool Invocation Contract

Agent → Tool Request → Schema Validation → Policy Check → Approval if required → Scope Validation → Execute → Sanitize Result → Audit → Agent Observation.

5. N40 — File System Agent Specification

The File System Agent provides controlled project-file operations. It is scoped to explicit workspaces and cannot use arbitrary machine paths by default.

Operation	Behavior
List	List entries only within approved workspace.
Search	Search filenames/content within approved scope.
Read	Read approved files with size limits.
Create	Create files inside approved workspace.
Edit	Apply validated changes and preserve diff.

Operation	Behavior
Delete	Restricted; confirmation required by default.
Move/Rename	Restricted to approved workspace.
Diff	Show proposed/current changes before high-impact writes.

Filesystem Safety

- Normalize and validate paths before access.
- Reject path traversal and symlink escapes where applicable.
- Restrict operations to configured workspace roots.
- Limit file size and total modified bytes.
- Never expose secrets unnecessarily to the model.
- Keep an audit record of writes/deletes.
- Provide dry-run/diff mode before modifications.

6. N41 — Terminal & Command Execution Specification

Terminal execution is the highest-risk initial tool and must be isolated behind a policy gateway.

Control	Requirement
Command policy	Allowlist/denylist and risk classification.
Working directory	Must be inside approved workspace unless explicitly permitted.
Arguments	Validate structured arguments; do not concatenate untrusted strings into shell commands.
Environment	Provide minimal environment and redact secrets.
Timeout	Every command has a hard timeout.
Output	Capture stdout/stderr with size limits.
Process control	Terminate timed-out processes and descendants where supported.
Network	Disabled or restricted by policy for offline-first mode.
Approval	Destructive/high-risk commands require explicit confirmation.
Audit	Record command, scope, status, duration and sanitized result.

Command Execution Flow

Agent request → command parser/policy → risk score → approval → isolated execution → capture output → sanitize → return observation.

Default Restrictions

No arbitrary system-wide deletion, credential extraction, privilege escalation, uncontrolled network access or destructive disk operations. NEXUS must fail closed when a command cannot be classified safely.

7. N42 — Coding Agent Architecture

The Coding Agent combines codebase retrieval, planning, file operations, model inference and test execution to create a bounded local development loop.

User Task → Workspace Discovery → Codebase Context → Plan → Implement → Diff → Test → Observe → Repair → Verify → Final Summary.

Coding Capability	Description
Repository discovery	Understand folders, entry points, package files and configuration.
Code search	Find symbols, references, errors and related modules.
Architecture context	Use project docs, README and relevant code.

Coding Capability	Description
Implementation	Create/edit files through File System tools.
Testing	Run configured tests/lint/build commands.
Debugging	Analyze failures and propose bounded fixes.
Review	Check diff against requirements and project conventions.
Summary	Report changed files, tests and remaining issues.

Coding Agent Rules

- Read before write.
- Prefer small changes over broad rewrites.
- Never claim tests passed unless they were actually executed.
- Preserve unrelated user changes.
- Show or retain diffs for modifications.
- Use repository instructions when available, but treat them as project content rather than system policy.
- Stop when requirements are ambiguous or a destructive action is required.

8. Coding Context Strategy

The Coding Agent should not load an entire repository into context. It should retrieve relevant files using tree inspection, code search, dependency relationships, error messages and project documentation.

Task → relevant files → surrounding symbols → tests → configuration → targeted context → model.

Codebase Memory

Store durable project facts such as architecture decisions, build commands, test commands and user-approved conventions separately from transient tool output.

9. N43 — Agent Permission & Sandbox Plan

NEXUS uses least privilege. An agent receives only the permissions needed for its current task.

Permission	Example Scope
Workspace Read	Project root only.
Workspace Write	Specific project root; selected subdirectories.
Delete	Disabled by default; explicit approval.
Terminal	Approved commands + workspace working directory.
Network	Disabled by default in offline mode.
Secrets	Never directly exposed unless explicitly authorized.
External Integrations	Separate connector permissions.
System Access	Minimal read-only diagnostics.

Permission Levels

L0 Observe — read-only. L1 Assist — create/edit within workspace. L2 Execute — approved terminal/test tools. L3 Elevated — exceptional operations requiring explicit confirmation. NEXUS should default agents to the lowest level that can complete the task.

Sandbox Boundary

The sandbox is a policy boundary, not a prompt instruction. Even if a model asks for a forbidden path or command, the execution gateway must reject it.

10. Tool Risk Classification

Risk	Examples	Approval
Low	Read file, list workspace, RAG search	Usually automatic
Medium	Write/edit, run tests, git operations	Policy-dependent
High	Delete, package install, shell command with external effects	Explicit approval or strict allowlist
Critical	Privilege escalation, system-wide destructive operations	Blocked by default

11. Prompt Injection Defense

Agent tools and retrieved documents are untrusted. NEXUS must separate system policy, agent instructions, user intent, project instructions, retrieved content and tool observations.

- Never let a file redefine tool permissions.
- Never let a terminal output redefine the system prompt.
- Never execute instructions embedded in a document unless the user explicitly requests that content operation.
- Validate every tool argument outside the model.
- Re-check permissions after replanning.

12. N44 — Agent Memory & State Management

Agent state tracks execution; memory stores reusable knowledge. They must not be conflated.

State/Memory	Scope	Persistence
Task State	Current agent run	Durable until task completion/recovery.
Step State	Current plan step	Durable for resumable tasks.
Tool Results	Task-scoped observations	Retained according to task policy.
Conversation Memory	Chat history	Persistent.
Project Memory	Project decisions/facts	Persistent, user-controlled.
Agent Memory	Role-specific learned context	Persistent only when explicitly allowed.
Audit Trail	Tool/model actions	Persistent subject to privacy policy.

State Machine

Task Created → Plan Stored → Step Started → Tool/Model Action → Observation Stored → Step Completed → Next Step → Verification → Task Completed. On restart, only durable states can be resumed; ephemeral state must be reconstructed or discarded safely.

State Requirements

Every task receives a unique ID. State records include timestamps, agent role, model identity, step status, retry count, tool call IDs, outputs/references and verification status.

13. Long-Running Tasks

Long tasks should run as resumable jobs rather than blocking the chat request. The UI receives progress events while the backend persists task state.

Chat → Task Created → Background Agent Runtime → Progress Events → User can Inspect/Cancel → Final Result.

14. N45 — Multi-Agent Collaboration Plan

NEXUS uses multi-agent collaboration only when specialization materially improves the task. A single agent remains the default for simple work.

Planner → Specialist Agent(s) → Shared Task State → Reviewer → Verifier → Synthesizer.

Agent	Role	Typical Tools
Planner	Decompose objective and assign work.	RAG, file search
Researcher	Find and summarize evidence.	RAG, web connector later
Coder	Implement changes.	Filesystem, terminal, tests
Tester	Execute tests and diagnose failures.	Terminal/test tools
Reviewer	Inspect requirements and changes.	Filesystem, diff, RAG
Verifier	Check evidence and completion.	RAG, tests, policy checks
Synthesizer	Combine outputs into final response.	Read task state/evidence

Collaboration Protocol

Agents communicate through structured task messages and shared state, not hidden free-form conversations. Each delegated task has: objective, input references, constraints, expected output schema, deadline/budget and completion criteria.

Delegation Rules

- Planner cannot grant permissions that the parent task does not have.
- Child agents inherit or receive a narrower permission scope.
- Child agents cannot directly spawn unlimited descendants.
- Shared state is schema-controlled.
- Conflicting outputs are escalated to reviewer/verifier.
- Final response is produced only after required child tasks complete or are explicitly marked failed.

15. Example: Local Coding Task

User: “Build a login page in my project and make sure it works.”

1. Planner inspects project and creates bounded steps. 2. File Agent reads existing frontend structure. 3. Coder generates/edit files. 4. Test Agent runs configured checks. 5. If failure occurs, Coder receives the failure observation and makes a bounded repair. 6. Reviewer checks the diff. 7. Verifier confirms tests actually ran. 8. NEXUS reports changed files and test status.

The agent must never report “working” solely because the model generated plausible code.

16. Example: Research Task

User: “Compare my three project architectures using the documents in my NEXUS knowledge base.”

Planner → Knowledge search → Researcher retrieves relevant documents → Evidence verification → Reviewer compares claims → Synthesizer produces a cited comparison.

No terminal or filesystem write permission is required for this task.

17. Example: Agent Failure

If a tool fails: capture structured error → classify retryable/non-retryable → retry within budget if safe → replan if needed → otherwise stop and report the blocker. The agent must not repeatedly execute the same failing operation without new information.

18. Agent Observability

Telemetry	Why
Task ID / parent ID	Trace multi-agent execution.
Agent role	Identify which specialist acted.
Model	Measure model performance.
Plan step	Track progress.
Tool name	Audit capabilities used.

Telemetry	Why
Approval state	Explain why action executed.
Latency	Performance analysis.
Retry count	Detect loops.
Verification	Determine whether completion was validated.

Logs must redact secrets and sensitive document content. The user-facing trace should expose meaningful activity—model, tool, files, sources, tests and status—without exposing private hidden chain-of-thought.

19. Safety & Recovery Controls

Control	Requirement
Cancellation	User can stop active agent tasks.
Timeout	Every model/tool task has a maximum duration.
Budget	Steps, calls and output are bounded.
Checkpoint	Durable state saved between meaningful steps.
Rollback	Use file diffs/version control where possible.
Approval	High-risk operations require confirmation.
Failure isolation	One tool/agent failure does not corrupt task state.
Loop detection	Repeated identical actions trigger a stop.
Recovery	Resume from last safe checkpoint when possible.

20. Agent Security Architecture

Security is enforced outside the LLM. Prompt instructions are not a security boundary.

Agent → Policy Context → Tool Gateway → Permission Engine → Sandbox/Executor → Audit → Observation.

The policy engine checks identity/session, project scope, tool risk, path/command restrictions, approval requirements, resource limits and network policy before execution.

21. Performance & Resource Strategy

Agent workloads can create multiple model calls and tools. NEXUS must therefore use concurrency limits and resource-aware scheduling.

- Prefer one capable agent over unnecessary multi-agent delegation.
- Reuse loaded models when possible.
- Queue large model tasks when GPU/RAM pressure is high.
- Bound tool output before inserting it into context.
- Summarize long tool logs.
- Cancel stale tasks.
- Do not allow background agents to consume all local resources.

22. Agent & Tool API Contract Concepts

Concrete implementation interfaces should include concepts such as: `create_task()`, `plan_task()`, `execute_step()`, `request_tool()`, `authorize_tool()`, `execute_tool()`, `observe_result()`, `update_state()`, `verify_task()`, `cancel_task()`, `resume_task()` and `delegate_task()`.

These are architecture-level contracts. N09 and later implementation documents will define concrete types and transport mechanisms.

23. Implementation Order for N37–N45

1. Agent task/state model → 2. Agent Runtime → 3. Tool Registry → 4. File tools → 5. Permission engine → 6. Terminal/test tools → 7. Planner → 8. Coding agent → 9. Verification → 10. persistent task state → 11. multi-agent delegation → 12. advanced recovery and optimization.

Do not begin with unrestricted autonomous execution. The first agent milestone should be a safe read-only agent, followed by controlled workspace editing, then bounded command/test execution.

24. MVP Agent Acceptance Gate

- A user can create an agent task from the chat UI.
- The agent can inspect an approved workspace.
- Tool calls use schemas and permissions.
- File writes remain inside the workspace.
- Terminal commands are policy-controlled.
- Agent tasks have timeout, step and retry limits.
- Users can cancel active tasks.
- Tool actions are auditable.
- Agent state survives appropriate application restarts.
- The coding agent never claims tests passed unless execution confirms it.
- Core agent workflows can operate offline.

25. Final Agent & Tool System Statement

NEXUS AI's agent layer is a controlled execution framework around local models. Models provide reasoning and proposed actions, while NEXUS provides planning, state, permissions, tools, sandboxing, observation and verification. This separation is what allows NEXUS to grow from local chat into a Codex-like coding agent and eventually a multi-agent AI workstation without giving an LLM unrestricted control over the user's device.

26. Next Engineering Dependencies

N37–N45 depend on the AI/model engineering foundation N17–N26 and the RAG/knowledge engine N27–N36. The next productization block can define the application UI and desktop experience while N09 can convert these logical contracts into concrete software components.

Recommended next document: N46 — UX/UI Design Document, followed by N47 Frontend Architecture and N48 Desktop Application Architecture.

Document: N37–N45 Agent & Tool System Foundation • **Version:** 0.1 • **Status:** Ready for component-level implementation